

[CS230] Improving Deep Neural Networks: Regularization (L2 / Weight Decay)

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-4. Neural Networks Basics - *Completed*

Chapter 5. Practical Aspects of Deep Learning (Current Unit)

- 5.1 Train / Dev / Test Sets Strategy - *Completed*
- 5.2 Bias vs Variance Analysis - *Completed*
- **5.3 Regularization (L1/L2)**
 - * Why Regularize? (Penalizing Complexity)
 - * L2 Regularization (Ridge) Formula
 - * Math: Why is it called "Weight Decay"?
 - * Implementation
- 5.4 Dropout Regularization - *Upcoming*

지난 시간 복습 및 연결

지난 시간에 우리는 모델이 **High Variance(과대적합)**라는 병에 걸렸음을 진단했습니다. 모델이 학습 데이터에만 너무 집착해서(암기해서), 실전 문제(Dev Set)를 못 푸는 상황입니다. 이제 처방전을 쓸 차례입니다. 과대적합을 치료하는 가장 전통적이고 강력한 항생제는 바로 '**정규화(Regularization)**'입니다. 정규화는 모델에게 "정답을 맞추되, 너무 꼼수(큰 가중치)는 쓰지 마라"라고 제약(Penalty)을 거는 것입니다.

1 Unit Overview

□ 핵심 목표

이 단원은 모델의 복잡도를 억제하여 일반화 성능을 높이는 **L2 정규화**를 집중적으로 다룹니다.

- **개념:** 비용 함수 J 에 가중치 크기($\|W\|^2$)에 비례하는 벌점을 추가합니다.
- **수학:** 역전파 과정에서 가중치가 스스로 줄어드는 **Weight Decay(가중치 감쇠)** 현상을 수식으로 증명합니다.
- **구현:** 정규화 항이 포함된 Forward 및 Backward 코드를 작성합니다.
- **비교:** L1 정규화(Lasso)와의 차이점(희소성)을 이해합니다.

2 Essential Terminology

용어	기호	핵심 의미
L2 Regularization	Ridge	가중치의 제곱합을 벌점으로 사용. $W \rightarrow 0$ (작아짐).
L1 Regularization	Lasso	가중치의 절댓값 합을 벌점으로 사용. $W = 0$ (사라짐/희소성).
Lambda	λ	정규화 강도. 클수록 모델이 단순해짐 (Underfitting 위험).
Weight Decay	-	매 업데이트마다 가중치가 일정 비율씩 감소하는 현상.
Frobenius Norm	$\ W\ _F^2$	행렬의 모든 원소를 제곱해서 더한 값.

3 Core Concepts: 벌점 시스템

3.1 1. The Idea of Penalty

우리의 목표는 비용 J 를 최소화하는 것입니다. 여기에 "가중치 W 가 커지면 벌점을 주겠다"는 새로운 규칙을 추가합니다.

$$J_{regularized}(W, b) = \underbrace{J_{original}(W, b)}_{\text{오차 (Cross Entropy)}} + \underbrace{\frac{\lambda}{2m} \sum_l \|W^{[l]}\|_F^2}_{\text{벌점 (L2 Penalty)}}$$

- λ (Lambda): 벌점의 강도입니다. 하이퍼파라미터입니다.
- m : 데이터 개수.
- $2m$: 미분할 때 제곱(²)이 내려와서 2와 약분되라고 미리 2로 나눠둡니다. (수학적 편의)

3.2 2. L1 vs L2 (Which one to use?)

- **L2 (Standard)**: 가중치를 0에 가깝게 만듭니다. 모든 특성을 골고루 사용하게 합니다. 딥러닝의 기본값(Default)입니다.
- **L1 (Sparse)**: 가중치를 완전히 0으로 만듭니다. 불필요한 특성을 제거 (Feature Selection) 하고 싶을 때 쓰지만, 미분이 까다로워 잘 안 씁니다.

4 Deep Dive: Why "Weight Decay"?

이 섹션은 L2 정규화를 왜 '가중치 감쇠'라고 부르는지 수학적으로 증명합니다.

□ 역전과 수식 유도 (수학적 증명)

비용 함수 J_{reg} 를 W 에 대해 미분해 봅시다.

$$\frac{\partial J_{reg}}{\partial W} = \frac{\partial J_{orig}}{\partial W} + \frac{\partial}{\partial W} \left(\frac{\lambda}{2m} W^2 \right)$$

$$dW_{reg} = dW_{orig} + \frac{\lambda}{m} W$$

(분모의 2가 미분되면서 사라졌습니다!)

이제 경사 하강법 업데이트 식에 대입합니다.

$$W_{new} = W - \alpha \cdot dW_{reg}$$

$$W_{new} = W - \alpha \left(dW_{orig} + \frac{\lambda}{m} W \right)$$

이 식을 W 로 묶으면 놀라운 결과가 나옵니다:

$$W_{new} = \underbrace{\left(1 - \frac{\alpha\lambda}{m} \right)}_{\text{Decay Factor } (<1)} W - \alpha \cdot dW_{orig}$$

결론: 매 업데이트마다 가중치 W 는 원래 학습 방향($-\alpha dW$)으로 가기 전에, 자기 자신의 크기를 $(1 - \frac{\alpha\lambda}{m})$ 비율만큼 줄입니다. 즉, 가만히 있어도 스스로 감소(Decay) 합니다.

5 Implementation: L2 Regularization

정규화는 Forward(비용 계산)와 Backward(기울기 계산) 양쪽에 모두 코드를 추가해야 합니다.

```
1 import numpy as np
2
3 class L2Regularizer:
4     def __init__(self, lamdb):
5         self.lamdb = lamdb
6
7     def compute_cost(self, cost_cross_entropy, parameters, m):
8         """
9         J_total = J_cross_entropy + (lambda / 2m) * sum(W^2)
10        """
11        L = len(parameters) // 2
12        L2_cost = 0
13
14        for l in range(1, L + 1):
15            W = parameters['W' + str(l)]
16            # Frobenius Norm 제곱 계산
17            L2_cost += np.sum(np.square(W))
18
19        L2_cost *= (self.lamdb / (2 * m)) # 2으로m 나눔 주의!
20
21        return cost_cross_entropy + L2_cost
22
23    def backward(self, dW_orig, W, m):
24        """
25        dW_reg = dW_orig + (lambda / m) * W
26        """
27        # 정규화 항의 기울기 추가 여기서는(으로m 나눔!)
28        dW_reg = dW_orig + ((self.lamdb / m) * W)
```

```

29         return dW_reg
30
31
32 # --- 실행 예제 ---
33 if __name__ == "__main__":
34     m = 1000
35     lambd = 0.7
36     reg = L2Regularizer(lambd)
37
38     # 가상의 W (Weight)
39     W = np.array([[0.5, -0.2], [0.1, 0.8]])
40     dW_orig = np.array([[0.01, 0.02], [-0.01, 0.05]]) # 원래 기울기
41
42     # 역전파 적용
43     dW_final = reg.backward(dW_orig, W, m)
44
45     print("Original dW:\n", dW_orig)
46     print("Regularized dW:\n", dW_final)
47     # dW 값이 W 부호 방향으로 조금 더 커짐 -> 를 W 쪽으로 더 세게 밀

```

Listing 1: L2 Regularization Implementation

6 FAQ & Pitfalls

편향(Bias) b 는 정규화 안 하나요? (오해 방지 가이드)

보통 안 합니다. b 는 함수의 모양(곡률)이 아니라 위치만 이동시킵니다. 따라서 모델의 복잡도에 큰 영향을 주지 않습니다. W 만 정규화해도 충분합니다.

Q. Lambda(λ) 값은 어떻게 정하나요?

A. 하이퍼파라미터입니다. 여러 값을 시도해보고 Dev Set의 오차가 가장 낮은 값을 찾아야 합니다. 보통 0.01, 0.001 처럼 로그 스케일로 탐색합니다.

다음 단계 (Next Step)

우리는 L2 정규화를 통해 가중치가 너무 커지는 것을 막아 과대적합을 억제했습니다.

하지만 때로는 더 과격한 방법이 필요할 때가 있습니다. 가중치를 줄이는 게 아니라, 아예 뉴런을 무작위로 꺼버리는(Shutdown) 방법입니다. "어떻게 뇌세포를 죽이는데 학습이 더 잘 되나요?" 다음 시간에는 딥러닝에서 가장 독특하고 강력한 정규화 기법인 [Regularization] Dropout (드롭아웃)을 배우겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **L2 Regularization:** 가중치 제곱합(W^2)을 비용 함수에 추가하여 큰 가중치에 벌점을 준다.
2. **Weight Decay:** 역전파 시 W 가 매번 조금씩 0을 향해 줄어든다.
3. **Effect:** W 가 작아지면 모델이 선형(Linear)에 가까워져 복잡도가 줄어든다. (과대적합 해결)
4. **Tip:** Cost 계산 시엔 $2m$, Gradient 계산 시엔 m 으로 나눈다.